

Claude API, Agent Skills & Subagents

From building Claude directly into your own applications, to teaching it reusable expertise, to delegating work into isolated context windows it can hand back cleanly.

Building with the Claude API

Messages, tool use, streaming — live integration demo

Agent Skills & Subagents

Reusable expertise, delegated work, a live code review

01 Building with the Claude API

Everywhere else this week, Claude has been the interface. Today it becomes a component — something your own application calls, with your own code deciding what happens with the response.

The request/response loop



Three capabilities that matter most in practice

CAPABILITY	WHAT IT DOES	TYPICAL USE
Messages	The core call — send conversation history, get a response back.	Any single request/response feature: summarization, drafting, Q&A.
Tool use	Claude decides when to call a function you define, and with what arguments.	Letting Claude look something up, run a calculation, or trigger an action in your system.
Streaming	Tokens arrive incrementally instead of all at once.	Anywhere a user is watching the response — chat UIs, live assistants.

```
// minimal request shape
const res = await fetch("https://api.anthropic.com/v1/messages", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    model: "claude-sonnet-5",
    max_tokens: 1000,
    messages: [{ role: "user", content: "Summarize this ticket." }]
  })
});
```

Model names and specific parameters move over time — confirm the current names in the docs at build time rather than hard-coding from memory.

02 Hands-On — Claude in Your Own App

1

Wire Claude into a small existing application, live

Take a working app the group already knows and add one Claude-powered feature end to end — a request, a response, and the app doing something with it (not just printing text to a console).

Goal: see the request/response loop as real code, not a diagram

2

Add a tool call

Extend the same feature so Claude can call one function you define — something simple, like looking up a record — and your app executes it and returns the result.

Goal: see tool use decided by the model, not hard-coded by you

03 Agent Skills

A Skill is a folder of instructions — a name, a description, and a body — that Claude reads on every session and pulls in automatically the moment a task matches. Unlike a system prompt, a Skill is scoped, shareable, and loads on demand rather than sitting in context all the time.

Skill vs. system prompt

SYSTEM PROMPT

- Always in context, every turn
- One flat block of instructions
- Set once per application or session

SKILL

- Only its name/description sit in context until it's needed
- Full body loads only when the task actually matches
- Packaged as a folder — reusable, shareable, versionable across a team

3

Build and trigger a custom Skill

Write a Skill for a real recurring task on your team — a report format, a code review checklist, a document template — and confirm Claude invokes it automatically when the matching request comes in.

Goal: a Skill the team can actually keep using after today

04 Subagents

A subagent is a separate Claude instance with its own context window and its own tool access, given one task and asked to report back a result. The point isn't a second opinion — it's keeping your main conversation from filling up with exploration and detail that only matters to the subtask.

When to reach for a subagent vs. a Skill vs. a hook

Skill

Something you want Claude to know how to do, applied inline in the main conversation.

Subagent

Something you want delegated to an isolated context so it doesn't crowd the main session.

Hook

Something you want to happen automatically, every time, regardless of what the model decides.



Use a subagent to review code committed in a branch

Point a review subagent at a real branch. It reads the diff, checks it against team conventions, and returns a summary and flagged concerns — without any of the diff content ever touching your main conversation's context.

Goal: a clean review summary back in the main session, detail kept isolated

Key takeaways from Day 3

- The API puts your own code in charge of the loop: send a request, Claude responds with text or a tool call, your app decides what happens next.
- Skills package reusable expertise that loads only when a task actually matches — lighter and more shareable than a standing system prompt.
- Subagents delegate work into an isolated context window, keeping the main conversation focused on the outcome, not the exploration.
- Skill, subagent, and hook answer three different questions: what Claude should know, what should run in isolation, and what must always happen.